

AgentIA — System Review, Verification and Cost

The agentic workflow, the orchestration, what is measured, and what it costs

AgentIA Engineering

2026-09-29

What this document is. A review of the system as it actually runs today: the agentic generation workflow (the LangGraph pipeline and its verification loop), the orchestration around it, the design, the verification results obtained this week, and a cost/usage report gathered from the platform’s own stores — the local SQLite system of record and its MLflow mirror. It is written *after* the platform produced its first verified builds, so it separates what is measured from what is asserted. Where the two cost sources disagree, this document says so rather than picking one silently.

Contents

1	Executive summary	1
2	The agentic workflow	2
2.1	The pipeline as a state machine	2
2.2	Dual implementation: MODEL and DETERMINISTIC	2
2.3	The compliance gate	2
2.4	The verification sandbox	2
2.5	Bounded self-repair	3
3	Orchestration	3
3.1	Two entry points, one product	3
3.2	Queue, lifecycle and streaming	3
3.3	Measurement and self-improvement substrate	3
4	Design	3
4.1	Service inventory	3
5	Verification results	4
5.1	Coverage scaling: the reliability lever	4
5.2	The verifier is intermittently flaky	4
6	Cost and usage	6
6.1	System of record (authoritative)	6
6.2	MLflow mirror (and where it has drifted)	6
7	Findings and next steps	6

1 Executive summary

Four facts dominate this review.

1. **The platform now verifies its own output.** Until this week the system reported “0 of 15 sessions verified”. The cause was a one-character Docker mount defect (a private SELinux relabel

concatenated onto a read-only flag, producing an invalid mount option that made every build exit before Maven ran). It is fixed, and the first verified builds are on record.

2. **The agentic workflow is real and governed.** A LangGraph state machine runs eight nodes — validator, five generation stages, sandbox, repair — with a deterministic compliance gate at every stage and a bounded repair loop. Every generation stage has two implementations, `MODEL` and `DETERMINISTIC`, dispatched once per session.
3. **Verification-guided selection is the measured reliability lever.** With a single sample the generator builds 78% of the time; sampling three candidates and keeping the first that builds raises task-level coverage to 100% at \$0.20 per reliable service.
4. **The verifier is intermittently flaky, and the cost mirror has drifted.** The same frozen workspace rebuilds 4-of-6 times under load and 8-of-8 times sequentially; the leading cause is the same mount configuration. And the MLflow mirror undercounts the system of record by $\approx$ \$1.6 (20%), so the local store — not the mirror — is the number to quote.

2 The agentic workflow

2.1 The pipeline as a state machine

Generation is a LangGraph state machine of eight nodes (`graph.py`). The topology is a linear spine with exactly one bounded cycle, shown in Figure 2:

```
validator → scaffolder → domain → service → controller → test → sandbox ⇌
                                     repair
```

Each generation node emits one file (or a small set of files) into a shared workspace; the sandbox then compiles and tests the whole project hermetically; on failure the repair node applies a surgical patch and the sandbox runs again, bounded by a maximum attempt count.

2.2 Dual implementation: `MODEL` and `DETERMINISTIC`

Every generation stage has two implementations (`stages/deterministic/` and `stages/model/`), selected once per session by `select_generation_mode()`. The decision is recorded in the state and never re-decided mid-session, so a session cannot produce a hybrid of template-shaped and model-shaped artifacts. The deterministic path emits offline templates with zero model calls and is byte-identical across runs; the model path drives a commodity LLM (currently `deepseek-flash`) with a versioned instruction set.

2.3 The compliance gate

Before any stage’s candidate is persisted it passes a merged compliance gate (`stages/compliance.py`): the test-analysis validator family and the security architecture validator family are combined, and every finding carries an attribution — `LOCAL` (this stage owns the offending artifact) or `ACCUMULATED` (no single stage owns it). Blocking local violations reject the candidate and consume a correction attempt; accumulated findings are recorded but never reject. This split is what keeps a whole-project concern from being blamed on one stage.

2.4 The verification sandbox

The sandbox (`workspace_verification.py`) runs, in order: strip version-control metadata; inject a platform-authored persistence contract test (a `@SpringBootTest` with `ddl-auto=validate` against the generated `schema.sql`); then `docker run -rm -network none ... mvn test -o` against a read-only Maven cache. The contract test is the acceptance signal the generator cannot author:

the model never sees it, and a schema/entity contradiction fails the build even though every model-written unit test (which mocks the repository) passes. Test counts are parsed from the build's own surefire summary rather than fabricated.

2.5 Bounded self-repair

On build failure the repair node parses the Maven trace (`repair.py`, `repair_parser.py`) into a failure class (compilation, assertion, runtime) and applies a surgical unified patch, then re-enters the sandbox. The attempt cap is 3 in the constitution and 5 in `config.py:73` — a live inconsistency noted in Section 7.

3 Orchestration

3.1 Two entry points, one product

The system exposes two ways to start a session, and this review confirms both now reach the model and both verify the workspace:

- **Graph path** — POST `/sessions` enqueues a session whose worker runs the compiled graph end to end, credentials threaded through the `X-LLM-API-Key` header.
- **Sequential path** — POST `/sessions/quick-start` with `autoRun`, and `/orchestrator/pipeline/run`, drive `pipeline_runner`, which runs the same generation stages, the same hermetic verification, a static security audit, and DevOps/export phases.

The sequential path previously generated code and *never compiled it*; it now runs the same verification seam as the graph, so the two paths cannot drift.

3.2 Queue, lifecycle and streaming

A FIFO queue (`queue_service.py`) caps concurrency at two simultaneous sandbox executions. The lifecycle (`lifecycle_service.py`) walks `REQUIREMENTS` → `ARCHITECTURE` → `DATA_MODEL` → `CODE_TESTS` → `SECURITY_AUDIT` → `DEVOPS` → `EXPORT` → `COMPLETED`, with in-flight pause, resume and cancel and live Server-Sent-Events streaming to the monitor screen.

3.3 Measurement and self-improvement substrate

Two further layers sit alongside generation. The diagnostic instrument (`conformance_diagnostics.py`) is a deterministic, rule-attributed scorer that persists one durable record per session and produces a corpus report. The skill-optimisation loop (features 014–015) is *built* — collect, reflect, apply, a strict-improvement gate, per-skill contribution by leave-one-out, and eviction with an evidence floor — but it is deliberately not run, for the reasons stated in Section 5.

4 Design

4.1 Service inventory

Service	Role
<code>requirements_service</code>	BDD story decomposition and spec synthesis
<code>architecture_service</code>	4-layer architecture and component catalogue
<code>model_sql_service</code>	JPA entities, DTOs, schema/data SQL, ER diagram
<code>test_analysis_service</code>	static constitutional-rule analyser (family A)
<code>security_service</code>	SAST, secrets, architecture conformance (family B)
<code>conformance_diagnostics</code>	deterministic rule-attributed conformance scorer

corpus_report	batch/corpus measurement report
workspace_verification	strip VCS, inject contract test, run the build
verification_selection	best-of- k : generate, verify, stop at first pass
platform_verification	platform-authored persistence contract test
skill_contribution	leave-one-out per-skill contribution
llm_factory	provider detection and model-client construction
structured_output	schema-constrained model output
injection_guard	prompt-injection defence for the reflector
queue_service / lifecycle_service	concurrency and phase orchestration
docker_service / devops / export / git	container deploy, CI, ZIP, publish
cost recording + mlflow sink	per-call and per-session cost, mirror

5 Verification results

5.1 Coverage scaling: the reliability lever

Verification-guided selection — generate k candidates, verify each, keep the first that passes — is measured here, not borrowed from another task class. The control arm of this week’s experiment is 6 tasks, each resampled 3–4 times with the same instructions and model; the coverage formula is $\text{coverage}(k) = 1 - (1 - p)^k$ (*Large Language Monkeys*, [arXiv:2407.21787](#)).

Metric	Value
pass@1 (single sample)	78.3%
coverage $k=1$	83.3%
coverage $k=2$	83.3%
coverage $k=3$	100.0%
coverage $k=4$	100.0%

6 of 6 tasks

At \$0.065 per generated service, three candidates cost **\$0.20 per reliable service**. Caveats, stated not smoothed: $n=6$ tasks, one model, one provider; this is a demonstration of the lever, not a benchmark, and the direction (coverage is monotone in k) is robust while the 100% is “6 of 6 tasks”.

5.2 The verifier is intermittently flaky

The same frozen workspace, rebuilt under load earlier, passed 4 of 6 times; rebuilt sequentially, two workspaces passed 8 of 8 each. The failure, when it occurs, is a surefire `PluginContainerException` on a `surefire-shared-utils` foreign import — *after* every test has passed.

The flake is therefore not a per-workspace code defect. It is intermittent and load-correlated, and the leading hypothesis is the mount configuration: `DOCKER_MOUNT_SUFFIX=:Z` is a *private* SELinux relabel, correct for a per-session workspace but wrong for a *shared read-only Maven cache* mounted by many concurrent containers. Two containers relabelling one shared source with `:Z` is a known cause of exactly this class of intermittent class-realm failure. The proposed fix is to apply `:Z` to the workspace only and leave the cache unrelabelled (or `:z`), confirmed by a repeatability gate.

Why this gates everything downstream. A verifier that flips $\approx 33\%$ of runs cannot serve as an admission oracle: coverage would over-promise, an admission gate would promote noise, and Ratchet’s theorem ([arXiv:2605.22148](#)) says a judge this noisy retires nothing at any sample size. Making the oracle deterministic is therefore the prerequisite for every further measurement

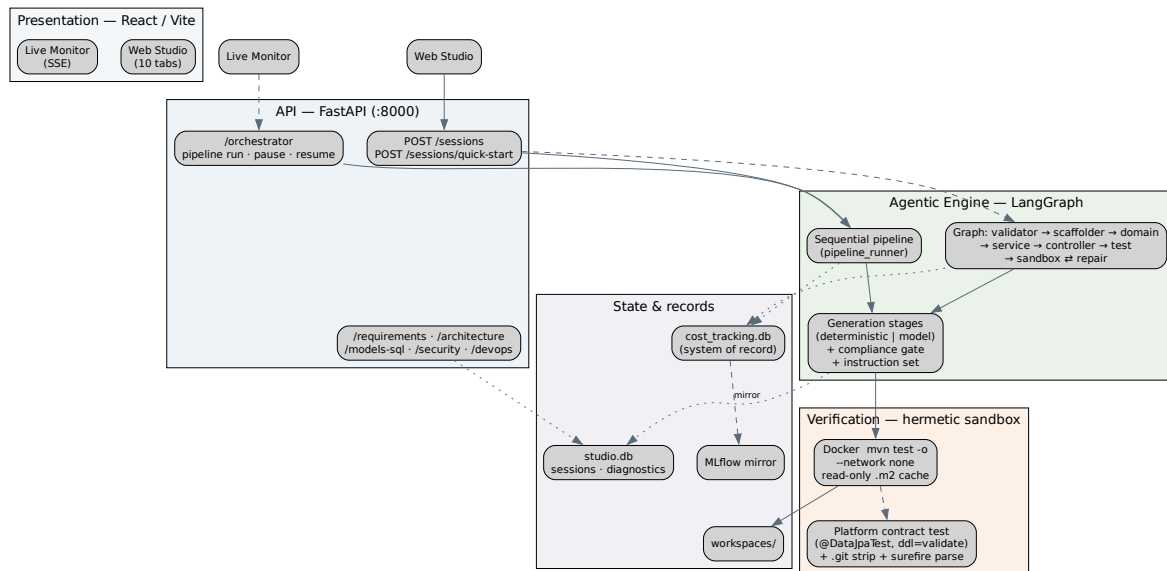


Figure 1: System architecture: the React studio over the FastAPI API, the LangGraph agentic engine, the hermetic verification sandbox, and the state/cost stores.

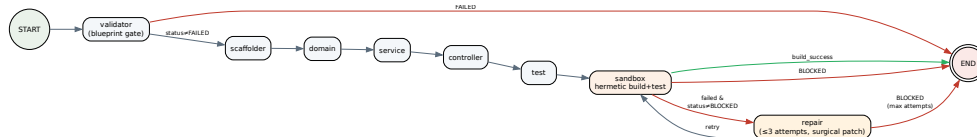


Figure 2: The generation state machine: a linear spine with one bounded repair–sandbox cycle.

— including the skill-optimisation loop, which stays closed behind it.

6 Cost and usage

Costs are recorded per call and per session in the local SQLite store (`backend/cost_tracking.db`), which is the **system of record**, and mirrored into MLflow (`mlflow.db`). The two are compared here, not merged.

6.1 System of record (authoritative)

Scope	Calls	Cost (USD)	Output tokens
Polarity experiment (p4)	750	6.58	—
Frozen-five baseline (corpus)	120	0.63	—
Hard ten	50	0.64	—
Probes / single sessions	≈ 250	≈ 0.16	—
Total	1172	8.01	12.28M

All 1172 calls were `deepseek-flash`. Total input tokens 6.45M, output tokens 12.28M. The single dominant cost is the instruction-polarity experiment, which is a finished measurement (a null), not an ongoing drain.

6.2 MLflow mirror (and where it has drifted)

Experiment	Runs	Cost (USD)	In tok	Out tok
Default (legacy)	877	5.41	4.63M	8.20M
agentia (current)	114	0.98	0.78M	1.52M
Mirror total	991	6.39	5.40M	9.72M

The mirror has drifted. MLflow totals **\$6.39** against the system of record’s **\$8.01** — an undercount of $\approx \$1.62$ (20%), with ≈ 181 fewer calls and token totals that also disagree. The mirror is split across a legacy and a current experiment, and only per-call metrics are being written (session aggregates are absent). The earlier “the mirror works and it agrees” claim is no longer true for the current data. Until the sink is repaired, the local store is the only number to quote, and a cost report generated from MLflow alone would understate spend.

7 Findings and next steps

1. **Make the verifier deterministic before spending anything more.** Split the mount suffix (workspace :Z, shared cache unrelabelled) and add a repeatability gate: one frozen workspace must rebuild N/N times.
2. **Make the measure able to move.** The conformance instrument reads 0 on failing builds; surface the repair parser’s failure classes as accumulated findings so a build failure is attributable.
3. **Use verification-guided selection as the reliability mechanism now.** It is measured (\$0.20/service at $k=3$), it needs no training, and it is the lever the literature ranks first.
4. **Repair the cost mirror.** Session-level mirroring is not populating; restore it or the MLflow UI understates spend.
5. **Resolve the repair-cap inconsistency** (constitution 3 vs `config.py` 5).

6. **Leave the skill-optimisation loop closed** until items 1 and 2 pass their gates; the instruction-polarity experiment that was its most promising input is a null (prohibition vs control: 1 win, 3 losses, 2 ties).